

# Laboratorio 8

## Teoría de la Computación

Respuestas: análisis de complejidad (Problemas 1–3, parte a),  
resultados de profiling (parte b) y Problemas 4 y 5

Notación: en C,  $n/2$  con enteros es división entera, es decir  $\lfloor n/2 \rfloor$ .  $T(n)$  es el número de veces que se ejecuta la instrucción del ciclo más interno. En todos los casos se suponen funciones positivas para  $n$  suficientemente grande.

# Problema 1

```
void function(int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 1; j+n/2 <= n; j++) {
            for (k = 1; k <= n; k = k*2) {
                counter++;
            }
        }
    }
}
```

## Parte a: complejidad en Big-O

### Primer ciclo (i)

$i = \lfloor n/2 \rfloor, \lfloor n/2 \rfloor + 1, \dots, n$   
 $\rightarrow n - \lfloor n/2 \rfloor + 1 = \lceil n/2 \rceil + 1$  iteraciones

Ejemplo con  $n = 10$ :  $i = 5, 6, 7, 8, 9, 10 \rightarrow 6$  iteraciones ( $\lceil 10/2 \rceil + 1 = 6$ ).

### Segundo ciclo (j)

Se despeja la condición del ciclo:

$j + \lfloor n/2 \rfloor \leq n$   
 $j \leq n - \lfloor n/2 \rfloor$   
 $j \leq \lceil n/2 \rceil$

$j = 1, 2, \dots, \lceil n/2 \rceil$   
 $\rightarrow \lceil n/2 \rceil$  iteraciones

No depende de  $i$ , así que se repite igual en cada vuelta del primer ciclo.

### Tercer ciclo (k)

$k$  se duplica en cada paso:

$k = 1, 2, 4, 8, \dots, 2^m$  con  $2^m \leq n$

$2^m \leq n \rightarrow m \leq \log_2 n \rightarrow m = \lfloor \log_2 n \rfloor$

Como  $m$  empieza en 0:  
 $\rightarrow \lfloor \log_2 n \rfloor + 1$  iteraciones

Ejemplo con  $n = 10$ :  $k = 1, 2, 4, 8 \rightarrow 4$  iteraciones ( $\lfloor \log_2 10 \rfloor + 1 = 3 + 1 = 4$ ). Este ciclo tampoco depende de  $i$  ni de  $j$ .

### Combinación de los ciclos anidados

Ningún ciclo depende de las variables de los otros, así que el total es el producto:

$T(n) = (\lceil n/2 \rceil + 1) \cdot \lceil n/2 \rceil \cdot (\lfloor \log_2 n \rfloor + 1)$

## Simplificación

Para  $n$  grande, los pisos, techos y el "+1" no cambian el orden de crecimiento:

$$T(n) \approx (n/2) \cdot (n/2) \cdot \log_2 n$$

$$T(n) \approx (1/4) \cdot n^2 \cdot \log_2 n$$

Se elimina la constante 1/4:

$$T(n) = O(n^2 \log n)$$

## Comprobación formal

**Cota superior.** Para  $n \geq 2$ :  $\lceil n/2 \rceil + 1 \leq n$ ,  $\lceil n/2 \rceil \leq n$  y  $\lfloor \log_2 n \rfloor + 1 \leq 2 \cdot \log_2 n$ .

$$T(n) \leq n \cdot n \cdot 2 \log_2 n = 2 \cdot n^2 \log_2 n \quad (c = 2, n_0 = 2)$$

**Cota inferior.** Cada factor es al menos  $n/2$ ,  $n/2$  y  $\log_2 n$ :

$$T(n) \geq (1/4) \cdot n^2 \log_2 n$$

Por lo tanto la cota es justa:  **$T(n) = \Theta(n^2 \log n)$** , y en particular  **$T(n) = O(n^2 \log n)$** .

## Valores exactos de counter

El programa implementado imprime el valor final de counter; coincide exactamente con la fórmula:

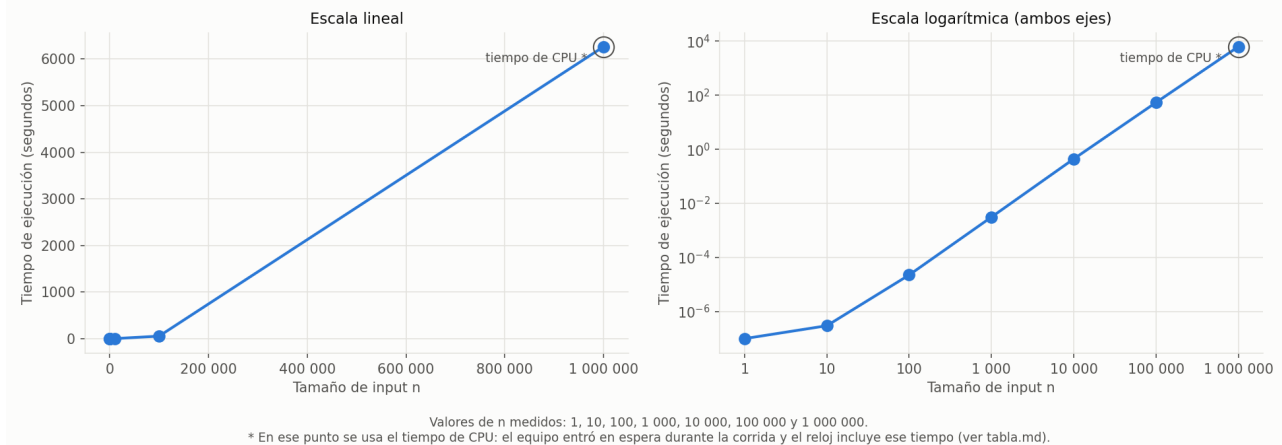
| $n$       | $\lceil n/2 \rceil + 1$ | $\lceil n/2 \rceil$ | $\lfloor \log_2 n \rfloor + 1$ | $T(n)$            |
|-----------|-------------------------|---------------------|--------------------------------|-------------------|
| 1         | 2                       | 1                   | 1                              | 2                 |
| 10        | 6                       | 5                   | 4                              | 120               |
| 100       | 51                      | 50                  | 7                              | 17 850            |
| 1 000     | 501                     | 500                 | 10                             | 2 505 000         |
| 10 000    | 5 001                   | 5 000               | 14                             | 350 070 000       |
| 100 000   | 50 001                  | 50 000              | 17                             | 42 500 850 000    |
| 1 000 000 | 500 001                 | 500 000             | 20                             | 5 000 010 000 000 |

## Parte b: profiling

Implementación en C (problema1/problema1.c), compilada con gcc -O0. El tiempo se mide con QueryPerformanceCounter alrededor de la llamada a function(n). Las corridas de menos de 1 s se repitieron 5 veces y se reporta la mediana.

| $n$       | Tiempo (s)          | Iteraciones del ciclo interno |
|-----------|---------------------|-------------------------------|
| 1         | 0.0000001           | 2                             |
| 10        | 0.0000003           | 120                           |
| 100       | 0.0000226           | 17 850                        |
| 1 000     | 0.0031283           | 2 505 000                     |
| 10 000    | 0.4426432           | 350 070 000                   |
| 100 000   | 54.1365546          | 42 500 850 000                |
| 1 000 000 | 6265.6406250 (CPU)* | 5 000 010 000 000             |

\* Para  $n = 1\,000\,000$  se reporta el **tiempo de CPU** (GetProcessTimes). Durante esa corrida el equipo entró en modo de espera unos 607 s y el programa quedó congelado; el reloj marcó 6873.10 s, que no corresponde al algoritmo. En los demás  $n$  el reloj y la CPU coinciden.

**Problema 1: tamaño de input vs. tiempo de ejecución (complejidad teórica  $O(n^2 \log n)$ )**

**Comparación con la teoría.** Al pasar de  $n = 10^5$  a  $n = 10^6$  el tiempo creció  $\times 116.4$  ( $6265.64 / 53.83$ ). La fórmula predice  $5\,000\,010\,000\,000 / 42\,500\,850\,000 \approx \times 117.6$ . En la gráfica log-log la recta tiene pendiente  $\approx 2$  (más el leve aporte del  $\log n$ ), lo que concuerda con  $O(n^2 \log n)$ .

## Problema 2

```
void function(int n) {
    if (n <= 1) return;
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Sequence\n");
            break;
        }
    }
}
```

### Parte a: complejidad en Big-O

#### Caso $n \leq 1$

La función retorna de inmediato. Solo se hace una comparación:  $T(n) = 1 = O(1)$ .

#### Primer ciclo (i), para $n \geq 2$

```
i = 1, 2, ..., n
→ n iteraciones
```

#### Segundo ciclo (j): efecto del break

En cada vuelta del ciclo externo, el ciclo interno hace:

```
j = 1
¿1 ≤ n?  sí (porque n ≥ 2)
printf("Sequence\n")
break    → sale del ciclo interno, sin llegar a j++ ni a j = 2
```

El ciclo interno **no** hace  $n$  iteraciones: hace **exactamente 1**, sin importar  $n$ . El break solo sale del ciclo interno; el externo continúa con  $i++$ .

#### Combinación de los ciclos

Cada iteración externa cuesta una cantidad constante  $c$  (inicializar  $j$ , una comparación, un printf y el break):

```
T(n) = Σ (i=1..n) c
T(n) = c · n
```

Se elimina la constante  $c$ :

```
T(n) = O(n)
```

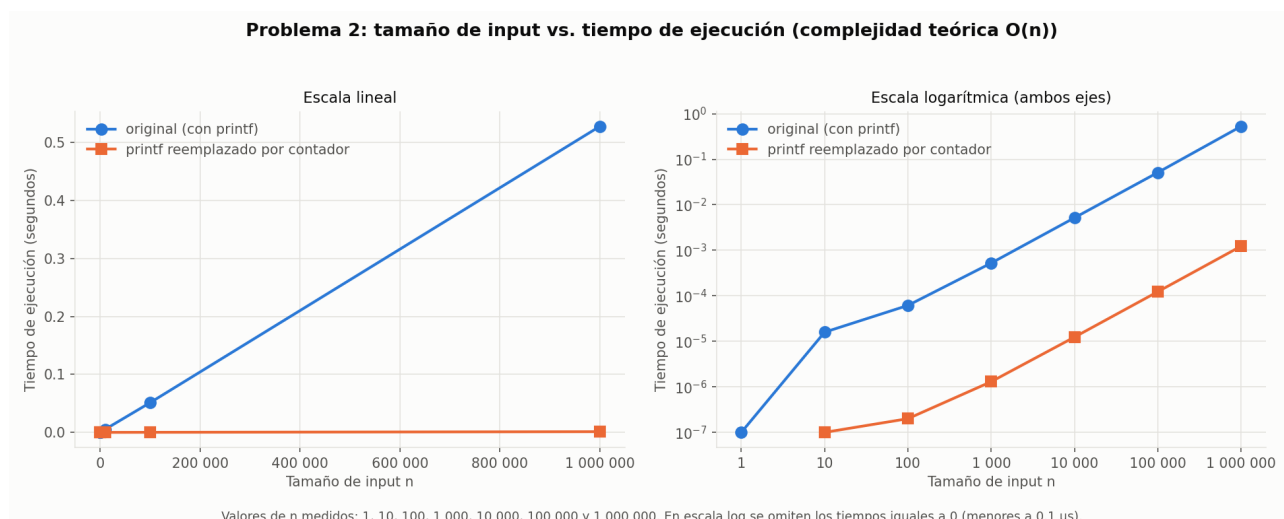
Aunque hay dos ciclos anidados, la complejidad **no** es  $O(n^2)$ : el break reduce el ciclo interno a costo constante. Para  $n \geq 2$  el printf se ejecuta exactamente  $n$  veces, así que  $c_1 \cdot n \leq T(n) \leq c_2 \cdot n$  y  **$T(n) = \Theta(n)$** .

## Parte b: profiling

Implementación en C (problema2/problema2.c). Se midieron dos versiones:

- **Original (con printf):** el código del enunciado sin cambios, con la salida estándar redirigida a NUL para no medir la velocidad de la consola.
- **printf reemplazado por contador:** la misma estructura de ciclos, pero el printf se cambia por contador++. Así se mide el costo de los ciclos sin el costo de entrada/salida.

| n         | Tiempo (s)<br>original (con printf) | Tiempo (s)<br>printf reemplazado por<br>contador | Iteraciones del<br>ciclo interno |
|-----------|-------------------------------------|--------------------------------------------------|----------------------------------|
| 1         | 0.0000001                           | 0.0000000                                        | 0                                |
| 10        | 0.0000159                           | 0.0000001                                        | 10                               |
| 100       | 0.0000615                           | 0.0000002                                        | 100                              |
| 1 000     | 0.0005241                           | 0.0000013                                        | 1 000                            |
| 10 000    | 0.0051749                           | 0.0000124                                        | 10 000                           |
| 100 000   | 0.0511648                           | 0.0001231                                        | 100 000                          |
| 1 000 000 | 0.5279288                           | 0.0012307                                        | 1 000 000                        |



**Comparación con la teoría.** En ambas versiones el tiempo se multiplica por  $\approx 10$  cada vez que  $n$  se multiplica por 10 (por ejemplo  $0.0512 \text{ s} \rightarrow 0.528 \text{ s}$  con printf entre  $n = 10^5$  y  $10^6$ ). En la gráfica log-log son rectas de pendiente  $\approx 1$ , que corresponde a  $O(n)$ .

## Problema 3

```
void function(int n) {
    int i, j;
    for (i = 1; i <= n/3; i++) {
        for (j = 1; j <= n; j += 4) {
            printf("Sequence\n");
        }
    }
}
```

### Parte a: complejidad en Big-O

#### Primer ciclo (i): efecto de $n/3$

$i = 1, 2, \dots, \lfloor n/3 \rfloor$   
 $\rightarrow \lfloor n/3 \rfloor$  iteraciones

El límite  $n/3$  solo divide el número de iteraciones por 3, una constante, así que el ciclo sigue siendo lineal. Si  $n < 3$ ,  $\lfloor n/3 \rfloor = 0$  y el ciclo no se ejecuta (con  $n = 1$  hay 0 impresiones).

#### Segundo ciclo (j): efecto de $j += 4$

$j = 1, 5, 9, \dots, 1 + 4t$  con  $1 + 4t \leq n$

$1 + 4t \leq n \rightarrow t \leq (n - 1)/4 \rightarrow t_{\max} = \lfloor (n - 1)/4 \rfloor$

Como  $t$  empieza en 0:

$\rightarrow \lfloor (n - 1)/4 \rfloor + 1$  iteraciones  $\approx n/4$

Ejemplo con  $n = 10$ :  $j = 1, 5, 9 \rightarrow 3$  iteraciones ( $\lfloor 9/4 \rfloor + 1 = 3$ ). El paso de 4 también divide el trabajo por una constante; el ciclo sigue siendo lineal.

#### Combinación de los ciclos anidados

El ciclo interno no depende de  $i$ , así que se multiplica:

$$T(n) = \lfloor n/3 \rfloor \cdot (\lfloor (n - 1)/4 \rfloor + 1)$$

#### Simplificación

$$T(n) \approx (n/3) \cdot (n/4)$$

$$T(n) \approx n^2/12$$

Se elimina la constante  $1/12$ :

$$T(n) = O(n^2)$$

#### Comprobación formal

**Cota superior.**  $\lfloor n/3 \rfloor \leq n/3$  y  $\lfloor (n-1)/4 \rfloor + 1 \leq (n+3)/4 \leq n$  para  $n \geq 1$ :

$$T(n) \leq (n/3) \cdot n = (1/3) \cdot n^2 \quad (c = 1/3, n_0 = 1)$$

**Cota inferior.** Para  $n \geq 6$ :  $\lfloor n/3 \rfloor \geq n/6$  y  $\lfloor (n-1)/4 \rfloor + 1 \geq n/4$ :

$$T(n) \geq n^2/24$$

Por lo tanto  $T(n) = \Theta(n^2)$ , y en particular  $T(n) = O(n^2)$ .

## Número exacto de printf

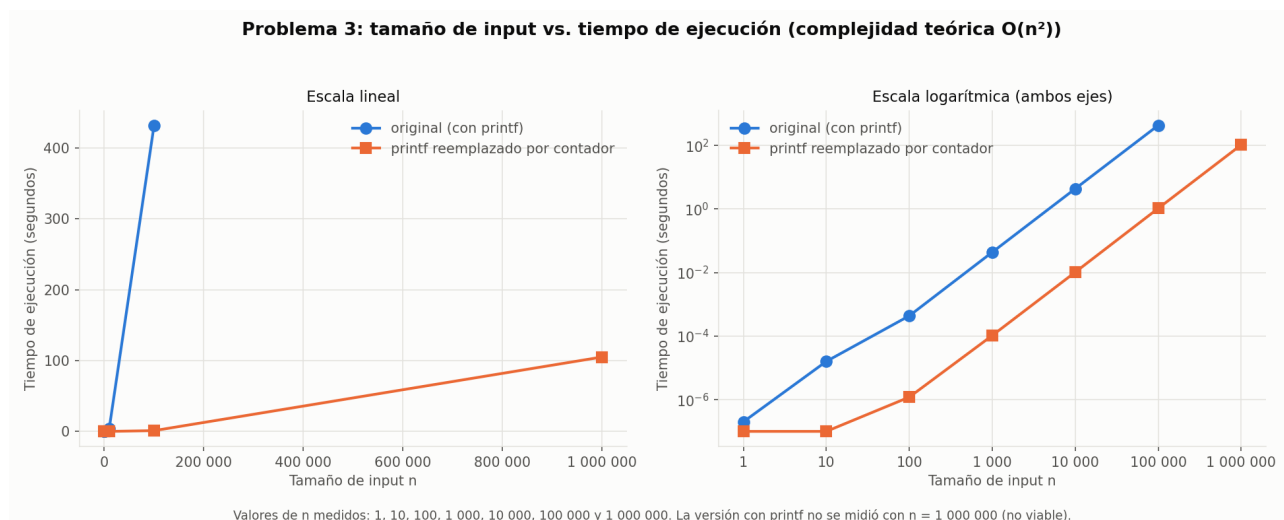
| n         | $\lfloor n/3 \rfloor$ | $\lfloor (n-1)/4 \rfloor + 1$ | T(n)           |
|-----------|-----------------------|-------------------------------|----------------|
| 1         | 0                     | 1                             | 0              |
| 10        | 3                     | 3                             | 9              |
| 100       | 33                    | 25                            | 825            |
| 1 000     | 333                   | 250                           | 83 250         |
| 10 000    | 3 333                 | 2 500                         | 8 332 500      |
| 100 000   | 33 333                | 25 000                        | 833 325 000    |
| 1 000 000 | 333 333               | 250 000                       | 83 333 250 000 |

## Parte b: profiling

Implementación en C (problema3/problema3.c), con las mismas dos versiones que el Problema 2: original con printf (salida a NUL) y printf reemplazado por contador.

| n         | Tiempo (s)<br>original (con printf) | Tiempo (s)<br>printf reemplazado por<br>contador | Iteraciones del<br>ciclo interno |
|-----------|-------------------------------------|--------------------------------------------------|----------------------------------|
| 1         | 0.0000002                           | 0.0000001                                        | 0                                |
| 10        | 0.0000160                           | 0.0000001                                        | 9                                |
| 100       | 0.0004361                           | 0.0000012                                        | 825                              |
| 1 000     | 0.0438704                           | 0.0001047                                        | 83 250                           |
| 10 000    | 4.3685680                           | 0.0103014                                        | 8 332 500                        |
| 100 000   | 431.9627476                         | 1.0515676                                        | 833 325 000                      |
| 1 000 000 | no ejecutado (no viable)            | 104.9981057                                      | 83 333 250 000                   |

La versión con printf y  $n = 1\,000\,000$  no se ejecutó: son 83 333 250 000 llamadas a printf. Cada una tarda  $\approx 0.5\ \mu\text{s}$  aun con la salida a NUL, lo que da unas 12 horas. La versión con contador sí se midió con todos los valores de  $n$ .



**Comparación con la teoría.** Cada vez que  $n$  se multiplica por 10, el tiempo se multiplica por  $\approx 100$  ( $\times 98.9$  con printf entre  $n = 10^4$  y  $10^5$ ;  $\times 99.8$  con contador entre  $n = 10^5$  y  $10^6$ ). En la gráfica log-log son rectas de pendiente  $\approx 2$ , que corresponde a  $O(n^2)$ .

## Problema 4

Unidad de costo: en las búsquedas, número de comparaciones con el elemento buscado  $x$ ; en Quick Sort, número de comparaciones entre elementos.

### 4.1 Búsqueda lineal

```
for i = 0 .. n-1:
    if A[i] == x: return i
return -1
```

Cada iteración hace 1 comparación. El costo depende de en qué posición está  $x$ .

#### Mejor caso

$x$  está en la primera posición,  $A[0]$ :

```
comparaciones = 1
T(n) = 1 =  $\Theta(1)$ 
```

#### Peor caso

$x$  está en la última posición, o no está en el arreglo. Se recorren los  $n$  elementos:

```
comparaciones = n
T(n) = n =  $\Theta(n)$ 
```

#### Caso promedio

Se supone que  $x$  está en el arreglo y que cada posición tiene probabilidad  $1/n$ . Si  $x$  está en la posición  $i$  (contando desde 1), se hacen  $i$  comparaciones:

$$\begin{aligned} E[T(n)] &= \sum_{i=1..n} i \cdot (1/n) \\ &= (1/n) \cdot n(n+1)/2 \\ &= (n+1)/2 \end{aligned}$$

Se elimina la constante  $1/2$ :

$$E[T(n)] = \Theta(n)$$

En promedio se revisa la mitad del arreglo, que sigue siendo lineal. Si  $x$  puede no estar (con probabilidad  $q$ ), el promedio es  $(1-q)(n+1)/2 + q \cdot n$ , que también es  $\Theta(n)$ .

| Caso     | Comparaciones | Complejidad |
|----------|---------------|-------------|
| Mejor    | 1             | $\Theta(1)$ |
| Promedio | $(n+1)/2$     | $\Theta(n)$ |
| Peor     | $n$           | $\Theta(n)$ |

### 4.2 Búsqueda binaria (arreglo ordenado)

```
izq = 0, der = n-1
while izq <= der:
    mid = (izq + der) / 2
    if A[mid] == x: return mid
    if A[mid] < x: izq = mid + 1
    else:         der = mid - 1
return -1
```

**Mejor caso**

x está justo en el centro: 1 comparación,  $T(n) = \Theta(1)$ .

**Peor caso**

x se encuentra al final de la búsqueda, o no está. En cada paso el rango se reduce a la mitad:

```
paso 1: n elementos
paso 2: n/2
paso 3: n/4
...
paso k: n/2^(k-1)
```

Termina cuando queda 1 elemento:

$$n/2^{(k-1)} = 1 \rightarrow 2^{(k-1)} = n \rightarrow k = \log_2 n + 1$$

Lo mismo con la recurrencia  $T(n) = T(n/2) + 1$ ,  $T(1) = 1$ , desenrollándola:

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &= T(n/4) + 2 \\ &= T(n/8) + 3 \\ &\dots \\ &= T(n/2^k) + k \end{aligned}$$

Con  $n/2^k = 1$ , es decir  $k = \log_2 n$ :

$$T(n) = 1 + \log_2 n = \Theta(\log n)$$

**Por qué aparece el logaritmo:**  $\log_2 n$  cuenta cuántas veces se puede dividir  $n$  entre 2 hasta llegar a 1. Cada comparación descarta la mitad de los elementos restantes, así que el número de comparaciones es ese número de divisiones.

**Caso promedio**

Las búsquedas forman un árbol binario de decisión balanceado: el elemento de la raíz se encuentra con 1 comparación, los 2 del nivel 2 con 2 comparaciones y, en general, los  $2^{(d-1)}$  elementos del nivel  $d$  con  $d$  comparaciones. Con  $n = 2^k - 1$  (árbol lleno de  $k$  niveles) y cada elemento igual de probable:

$$E[T(n)] = (1/n) \cdot \sum_{(d=1..k)} d \cdot 2^{(d-1)}$$

$$\text{Identidad: } \sum_{(d=1..k)} d \cdot 2^{(d-1)} = (k-1) \cdot 2^k + 1$$

$$\begin{aligned} E[T(n)] &= [(k-1) \cdot 2^k + 1] / n \\ &= [(k-1)(n+1) + 1] / n \\ &\approx k - 1 \\ &= \log_2(n+1) - 1 \end{aligned}$$

Ejemplo con  $n = 7$  ( $k = 3$ ):  $(1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4) / 7 = 17/7 \approx 2.43$  comparaciones. Por lo tanto  $E[T(n)] = \Theta(\log n)$ . Más de la mitad de los elementos están en el último nivel, por eso el promedio queda a solo  $\sim 1$  comparación del peor caso.

| Caso     | Comparaciones                | Complejidad      |
|----------|------------------------------|------------------|
| Mejor    | 1                            | $\Theta(1)$      |
| Promedio | $\approx \log_2(n+1) - 1$    | $\Theta(\log n)$ |
| Peor     | $\lceil \log_2 n \rceil + 1$ | $\Theta(\log n)$ |

### 4.3 Quick Sort

```
quicksort(A, izq, der):
    if izq < der:
        p = particion(A, izq, der) // pivote en su lugar final
        quicksort(A, izq, p-1)
        quicksort(A, p+1, der)
```

La partición de un subarreglo de  $m$  elementos compara cada elemento con el pivote:  $m - 1$  comparaciones, es decir  $\Theta(m)$ . El costo total depende de cómo divide el pivote el arreglo:

$$T(n) = T(\text{izquierda}) + T(\text{derecha}) + (n - 1)$$

#### Peor caso: el pivote siempre es el mínimo o el máximo

Ocurre, por ejemplo, si el arreglo ya está ordenado y se toma el último elemento como pivote. Un lado queda vacío y el otro con  $n - 1$  elementos:

$$T(n) = T(n-1) + T(0) + (n-1)$$

$$T(n) = T(n-1) + (n-1)$$

Desenrollando:

$$T(n) = (n-1) + (n-2) + \dots + 1 + 0$$

$$= n(n-1)/2$$

Se eliminan la constante  $1/2$  y el término de menor orden:

$$T(n) = \Theta(n^2)$$

#### Mejor caso: el pivote siempre es la mediana

Cada partición deja dos mitades iguales:

$$T(n) = 2T(n/2) + (n-1)$$

Árbol de recursión:

|                                                  |                                     |
|--------------------------------------------------|-------------------------------------|
| nivel 0: 1 subproblema de tamaño $n$             | → costo $\approx n$                 |
| nivel 1: 2 subproblemas de tamaño $n/2$          | → costo $\approx 2 \cdot (n/2) = n$ |
| nivel 2: 4 subproblemas de tamaño $n/4$          | → costo $\approx n$                 |
| ...                                              |                                     |
| nivel $j$ : $2^j$ subproblemas de tamaño $n/2^j$ | → costo $\approx n$                 |

Los subproblemas llegan a tamaño 1 cuando  $n/2^j = 1 \rightarrow j = \log_2 n$

$$T(n) \approx n \cdot \log_2 n = \Theta(n \log n)$$

Lo confirma el Teorema Maestro:  $a = 2$ ,  $b = 2$ ,  $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$ , caso 2  $\rightarrow \Theta(n \log n)$ .

#### Caso promedio: pivote aleatorio (o entrada en orden aleatorio)

El pivote termina en la posición  $i$  con probabilidad  $1/n$ . Si queda en la posición  $i$ , deja  $i - 1$  elementos a la izquierda y  $n - i$  a la derecha:

$$T(n) = (n-1) + (1/n) \cdot \sum_{i=1..n} [T(i-1) + T(n-i)]$$

Cada  $T(k)$  aparece dos veces:

$$T(n) = (n-1) + (2/n) \cdot \sum_{k=0..n-1} T(k)$$

Multiplicando por  $n$ :

$$n \cdot T(n) = n(n-1) + 2 \cdot \sum_{k=0..n-1} T(k) \quad (1)$$

La misma ecuación para  $n-1$ :

$$(n-1) \cdot T(n-1) = (n-1)(n-2) + 2 \cdot \sum_{k=0..n-2} T(k) \quad (2)$$

Restando (1) - (2):

$$n \cdot T(n) - (n-1) \cdot T(n-1) = 2(n-1) + 2 \cdot T(n-1)$$

$$n \cdot T(n) = (n+1) \cdot T(n-1) + 2(n-1)$$

Dividiendo entre  $n(n+1)$ :

$$T(n)/(n+1) = T(n-1)/n + 2(n-1)/(n(n+1))$$

Con  $2(n-1)/(n(n+1)) = 4/(n+1) - 2/n$  y  $S(n) = T(n)/(n+1)$ ,  $S(1) = 0$ :

$$S(n) = \sum_{k=2..n} [4/(k+1) - 2/k] = 2 \cdot H_n + 4/(n+1) - 4$$

donde  $H_n = 1 + 1/2 + \dots + 1/n$  (número armónico)

$$T(n) = (n+1) \cdot S(n) = 2(n+1) \cdot H_n - 4n$$

Verificación: con  $n=2$  la fórmula da  $2 \cdot 3 \cdot (3/2) - 8 = 1$  y el cálculo directo da  $(2-1) + (2/2)(0+0) = 1$ . Con  $n=3$  da  $2 \cdot 4 \cdot (11/6) - 12 = 8/3$  y el directo  $2 + (2/3)(0+0+1) = 8/3$ .

Como  $H_n \approx \ln n$ :

$$T(n) \approx 2n \cdot \ln n \approx 1.39 \cdot n \cdot \log_2 n = \Theta(n \log n)$$

### Cómo influye el pivote

- **Pivote balanceado:** cada nivel de recursión cuesta  $\Theta(n)$  y hay  $\Theta(\log n)$  niveles  $\rightarrow \Theta(n \log n)$ .
- **Pivote extremo:** solo se elimina un elemento por nivel; hay  $n$  niveles con costos  $n-1, n-2, \dots \rightarrow \Theta(n^2)$ .
- Aun particiones desbalanceadas, como  $1/10$  y  $9/10$ , dan profundidad  $\log_{10/9} n = \Theta(\log n)$  y costo total  $\Theta(n \log n)$ . Por eso el caso promedio se parece al mejor caso.
- Elegir el pivote al azar, o con la mediana de tres, hace muy improbable el peor caso, aunque no lo elimina.

| Caso     | Recurrencia                      | Comparaciones        | Complejidad        |
|----------|----------------------------------|----------------------|--------------------|
| Mejor    | $T(n) = 2T(n/2) + (n-1)$         | $\approx n \log_2 n$ | $\Theta(n \log n)$ |
| Promedio | $T(n) = (n-1) + (2/n) \sum T(k)$ | $2(n+1)H_n - 4n$     | $\Theta(n \log n)$ |
| Peor     | $T(n) = T(n-1) + (n-1)$          | $n(n-1)/2$           | $\Theta(n^2)$      |

## Problema 5

**a) Si  $f(n) = \Theta(g(n))$  y  $g(n) = \Theta(h(n))$ , entonces  $h(n) = \Theta(f(n))$ .**

**Respuesta: VERDADERO.**

Por definición, existen constantes positivas tales que:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{para } n \geq n_1 \quad (1)$$

$$c_3 \cdot h(n) \leq g(n) \leq c_4 \cdot h(n) \quad \text{para } n \geq n_2 \quad (2)$$

**Transitividad ( $f = \Theta(h)$ ).** Sustituyendo (2) en (1), para  $n \geq n_0 = \max(n_1, n_2)$ :

$$f(n) \geq c_1 \cdot g(n) \geq c_1 \cdot c_3 \cdot h(n)$$

$$f(n) \leq c_2 \cdot g(n) \leq c_2 \cdot c_4 \cdot h(n)$$

$$\rightarrow (c_1 c_3) \cdot h(n) \leq f(n) \leq (c_2 c_4) \cdot h(n) \quad \text{es decir } f = \Theta(h)$$

**Simetría ( $h = \Theta(f)$ ).** Despejando  $h(n)$  de ambas desigualdades:

$$f(n) \leq c_2 c_4 \cdot h(n) \rightarrow h(n) \geq (1/(c_2 c_4)) \cdot f(n)$$

$$f(n) \geq c_1 c_3 \cdot h(n) \rightarrow h(n) \leq (1/(c_1 c_3)) \cdot f(n)$$

$$\rightarrow (1/(c_2 c_4)) \cdot f(n) \leq h(n) \leq (1/(c_1 c_3)) \cdot f(n) \quad \text{para } n \geq n_0$$

Las constantes  $1/(c_2 c_4)$  y  $1/(c_1 c_3)$  son positivas, así que  **$h(n) = \Theta(f(n))$** . ■ En general,  $\Theta$  es una relación de equivalencia (reflexiva, simétrica y transitiva). Ejemplo:  $f = 3n^2 + n$ ,  $g = n^2$ ,  $h = 5n^2$ ; las tres son  $\Theta(n^2)$  y cada una es  $\Theta$  de las otras.

**b) Si  $f(n) = O(g(n))$  y  $g(n) = O(h(n))$ , entonces  $h(n) = \Omega(f(n))$ .**

**Respuesta: VERDADERO.**

$$f(n) \leq c_1 \cdot g(n) \quad \text{para } n \geq n_1$$

$$g(n) \leq c_2 \cdot h(n) \quad \text{para } n \geq n_2$$

**Paso 1, transitividad de  $O$ .** Para  $n \geq n_0 = \max(n_1, n_2)$ :

$$f(n) \leq c_1 \cdot g(n) \leq c_1 \cdot c_2 \cdot h(n) \rightarrow f(n) = O(h(n)) \quad \text{con } c = c_1 c_2$$

**Paso 2, simetría transpuesta de  $O$  y  $\Omega$ .** Despejando  $h(n)$ :

$$f(n) \leq c_1 c_2 \cdot h(n) \rightarrow h(n) \geq (1/(c_1 c_2)) \cdot f(n) \quad \text{para } n \geq n_0$$

Esa es la definición de  $\Omega$  con constante  $c' = 1/(c_1 c_2) > 0$ , así que  **$h(n) = \Omega(f(n))$** . ■

Ejemplo:  $f = n$ ,  $g = n^2$ ,  $h = n^3$ . Se cumple  $n = O(n^2)$  y  $n^2 = O(n^3)$ , y en efecto  $n^3 = \Omega(n)$ . La conclusión no se puede reforzar a  $\Theta$ : con el mismo ejemplo,  $n^3 \neq O(n)$ , así que  $n^3 \neq \Theta(n)$ .

**c)  $f(n) = \Theta(n^2)$ , donde  $f(n)$  es el tiempo de ejecución de  $A(n)$ .**

```
def A(n):
    atupla = tuple(range(0, n))
    S = set()
    for i in range(0, n):
        for j in range(i + 1, n):
            S.add(atupla[i:j])
```

**Respuesta: FALSO.** En realidad  $f(n) = \Theta(n^3)$ .

### Costo de cada parte

- `tuple(range(0, n))` construye una tupla de  $n$  elementos:  $\Theta(n)$ .
- `set()` crea un conjunto vacío:  $\Theta(1)$ .

### Número de iteraciones de los ciclos

```
i = 0, 1, ..., n-1
j = i+1, ..., n-1      →  n-1-i iteraciones

Σ (i=0..n-1) (n-1-i) = (n-1) + (n-2) + ... + 0 = n(n-1)/2
```

Si cada iteración costara  $O(1)$ , el total sería  $\Theta(n^2)$  y la afirmación sería verdadera. **Pero cada iteración no cuesta  $O(1)$ .**

### Costo de una iteración: `S.add(atupla[i:j])`

Sea  $L = j - i$  la longitud del slice:

1. `atupla[i:j]` **crea una tupla nueva** copiando  $L$  elementos:  $\Theta(L)$ .
2. `S.add(...)` debe calcular el **hash de la tupla**. En CPython el hash de una tupla recorre todos sus elementos (y no se guarda en caché):  $\Theta(L)$ .
3. La inserción en el set es  $O(1)$  esperado. Todos los slices son distintos (cada par  $(i, j)$  da la secuencia distinta  $i, \dots, j-1$ ), así que no hay comparaciones de igualdad completas, salvo colisiones de hash poco probables.

Por lo tanto cada iteración cuesta  $\Theta(j - i)$ .

### Costo total

$$f(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j - i) \\ = \sum_{i=0}^{n-1} \sum_{L=1}^{n-1-i} L$$

Con  $m = n - 1 - i$  ( $m = 0, 1, \dots, n-1$ ):

$$f(n) = \sum_{m=0}^{n-1} m(m+1)/2 \\ = (1/2) \cdot [ \sum m^2 + \sum m ] \\ = (1/2) \cdot [ (n-1)n(2n-1)/6 + (n-1)n/2 ] \\ = (1/2) \cdot (n-1)n(2n-1+3)/6 \\ = (n-1) \cdot n \cdot (n+1) / 6 \\ = (n^3 - n) / 6$$

Más el  $\Theta(n)$  de construir `atupla`:

$$f(n) = (n^3 - n)/6 + \Theta(n) = \Theta(n^3)$$

### Por qué no es $\Theta(n^2)$

Si fuera  $f(n) = O(n^2)$ , existiría  $c$  tal que  $(n^3 - n)/6 \leq c \cdot n^2$  para todo  $n$  grande. Dividiendo entre  $n^2$ :

$$(n - 1/n)/6 \leq c$$

El lado izquierdo crece sin límite, así que ninguna constante  $c$  funciona. Entonces  $f(n) \neq O(n^2)$  y, por lo tanto,  **$f(n) \neq \Theta(n^2)$ .**

La afirmación solo cuenta las  $n(n-1)/2$  iteraciones de los ciclos y omite que el slice y su hash cuestan un tiempo proporcional a la longitud de la tupla. La cota  $\Omega(n^2)$  sí se cumple, pero la cota  $O(n^2)$  no. Además, el set termina guardando  $(n^3 - n)/6$  elementos en total, así que la memoria también es  $\Theta(n^3)$ .

## Resumen del Problema 5

| Inciso | Respuesta | Razón                                                                              |
|--------|-----------|------------------------------------------------------------------------------------|
| a      | Verdadero | $\Theta$ es transitiva y simétrica                                                 |
| b      | Verdadero | $O$ es transitiva, y $f = O(h) \Leftrightarrow h = \Omega(f)$                      |
| c      | Falso     | Cada iteración cuesta $\Theta(j-i)$ ; $f(n) = (n^3-n)/6 + \Theta(n) = \Theta(n^3)$ |